Hosting a React Frontend on Amazon S3

From Local Production Build to Global Cloud Web Hosting

Agenda: Why static hosting on S3, preparing the React build, S3 bucket creation, public access permissions, bucket policies, and testing the live endpoint.



Made with GAMMA

Why Choose Amazon S3 for Static Hosting?



Definition: Amazon Simple Storage Service (S3) provides highly durable, scalable object storage.



Benefits for Frontends: Zero server management, infinite scalability, automatic high availability across multiple facilities, and extremely low cost (pay-per-request).



Ideal Workloads: Single Page Applications (SPAs) built with React, Vue, or Angular, alongside HTML/CSS/JS websites.

Made with GAMMA

Anatomy of a React Production Build

THE DEVELOPMENT STATE

The Development State: React runs on a local development server (e.g., npm run dev) with hot-reloading.

THE PRODUCTION STATE

The Production State: Running `npm run build` bundles, minifies, and compiles your code into optimized static assets (HTML, CSS, JS) inside a `dist/` or `build/` folder.

Why S3 Fits: S3 natively serves static HTML files and bundled assets without needing an underlying EC2 server or NGINX instance.

Made with GAMMA

Step 1: Creating Your S3 Bucket

01

Action: Open the AWS S3 Console and click “Create bucket”.

02

Naming Rule: Bucket names must be globally unique across all AWS accounts worldwide.

03

Region Selection: Choose the AWS Region closest to your target audience to minimize network latency.

Made with GAMMA

Step 2: Configuring Public Access Settings

01

Default Security: By default, AWS blocks all public access to S3 buckets for security.

02

Modification for Web Hosting: Uncheck “Block all public access” to allow worldwide web users to view your website.

03

Acknowledgement: Acknowledge the warning prompt confirming that the bucket will become public.

Made with GAMMA

Step 3: Enabling Static Website Hosting

01

Location: Go to the bucket's "Properties" tab and scroll down to "Static website hosting".

02

Configuration: Select "Enable".

03

Document Mapping: Set both the Index document and Error document to index.html (crucial for Single Page Applications to handle client-side routing properly).

Made with GAMMA

Step 4: Writing & Applying a Public Bucket Policy

Purpose: S3 requires an explicit JSON policy to grant read permissions to public internet users.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket-name/*"
    }
  ]
}
```

Made with GAMMA

Step 5: Uploading Your React Build Artifacts

01

Action: Go to the “Objects” tab in your bucket and click “Upload”.

02

What to Upload: Drag and drop the entire contents of your React dist/ or build/ folder (index.html and the assets/ folder).

03

Execution: Confirm the upload and verify files are listed in the bucket root.

Made with GAMMA

Step 6: Accessing Your Live Website Endpoint



Finding the URL: Go back to “Properties” → “Static website hosting” to find your unique S3 website endpoint URL.



Live Testing: Click or copy the URL into any web browser to view your React application running live on AWS!

Made with GAMMA

Summary & Production Best Practices

Limitations of Basic S3: S3 alone uses HTTP and lacks advanced routing rules for complex SPAs.

Production Architecture Upgrade: Combine **Amazon S3** (storage) with **Amazon CloudFront** (CDN for HTTPS and global edge caching) and **Route 53** (custom domain name).



Made with GAMMA